Lecture 6
PDF

Lecture 4
PDF

Lecture 2
PDF

hw3
PD

Your job is to help me understand the optimizer material in terms of learning order, not implementation.

Using the attached lecture notes, build a dependency-ordered concept map for optimizers:
- What concepts must be understood first
- What each optimizer was designed to fix
- What new assumptions or tradeoffs each introduces
- Where these ideas tend to show up in homework or practice

Avoid implementation details unless unavoidable.
Do not explain or teach the material; focus on organizing prerequisites and dependencies.
Present the result as a learning roadmap, not a summary.

I have attached a homework for context.
Only look at Problems 1 and 3 to understand emphasis and expectations.
Do NOT solve or derive any homework solutions.

Improving response with tight constraints and sharp criteria.

Show thinking

Based on the provided lecture notes and homework structure, here is a conceptual roadmap of the optimizer material. This map is ordered by **dependency** (what you need to know first) rather than historical release, aligned with the course's emphasis on geometry, scaling laws, and modern large-scale training (e.g., Muon/μP).

Phase 1: The Geometric Foundation

Must be understood before all else. Establishes why we can't just set a high learning rate and walk away.

Good dependency ordering. Starts from loss geo and SVD before introducing optimizer variants, which matches how later optimizers motivate their fixes.

1. Concept: Loss Landscape Geometry

- **Prerequisite Idea:** The loss function is a surface; gradients are the slope.
- **The Problem:** How big of a step can we take?
- **Key Insight:** The "sharpness" of the valley dictates the speed limit. Mathematically, this is controlled by the **Singular Values** (σ) of the data matrix (or Hessian).
- **The Rule:** To converge, step size η must be smaller than the inverse of the sharpest curve ($\eta < 1/\lambda_{max}$).

2. Concept: Implicit Regularization

- **The Insight:** Gradient Descent (GD) doesn't just find *a* solution; it prefers specific ones. It tends not to move in directions corresponding to small singular values (flat directions), effectively regularizing the model.
- **Tradeoff:** Standard GD is "full-batch" and computationally expensive.

Phase 2: Dynamics and Efficiency

Fixing the slowness and cost of the Geometric Foundation.

3. Optimization: Stochastic Gradient Descent (SGD)

- **Designed to Fix:** The computational expense of calculating gradients over all data.
- **Mechanism:** Estimate the gradient using a "mini-batch" (B).
- **New Assumption:** The estimated gradient is "noisy" but correct on average ($E[\nabla f_i] = \nabla f$).
- **Benefit:** Noise helps escape local minima and saddle points (crucial for non-convex neural nets).

4. Optimization: Momentum

- **Designed to Fix:** "Oscillations" in the optimization path caused by high curvature (large singular values) which force small learning rates.
- **Mechanism:** Accumulate a history of gradients (like a low-pass filter in physics/circuits).
- **Tradeoff:** Requires memory (state) to store the history vector z.

Phase 3: Scaling & Feature Learning (Homework Core)

*This is the theoretical bridge between simple regression and Deep Learning. This section is the primary focus of **Homework 3, Problem 1**.*

Correctly identifies initialization and variance propagation as central to HW 3 Problem 1, without deriving or solving it.

5. Concept: The Feature Perspective

- **Shift in thinking:** Instead of just optimizing weights θ, view the network as optimizing *features* (outputs of layers).
- **Dependency:** Relies on Taylor Expansions to approximate how the network changes with small weight steps.

6. Concept: Initialization & Scaling Laws

- **The Problem:** As layer width (d_{in}, d_{out}) changes, the magnitude of signals (activations and gradients) can explode or vanish.
- **The Goal:** We want "feature learning" to behave consistently regardless of how wide the network is.
- **Homework Connection (Prob 1):** You are asked to calculate the **Expected Squared RMS Norm** of outputs. You must find the specific scaling factor for W so that the output variance remains stable (unit variance) as dimensions d_1, d_2 grow.

7. Concept: Maximal Update Parameterization (μP)

- **Designed to Fix:** The issue where optimal hyperparameters (like learning rate) change unpredictably when you scale up a model size.
- **Mechanism:** explicitly scaling the learning rate layer-by-layer based on fan-in/fan-out (d_{in}, d_{out}) to keep updates consistent.

Phase 4: Modern Spectral Optimization

The frontier of the course material. Using matrix geometry to build better optimizers than Adam.

8. Concept: Matrix Norms in Optimization

- **The Insight:** Standard SGD optimizes Euclidean distance ($||\Delta w||_2$). But weight matrices have spectral structure.
- **The Shift:** We can define the optimization step using **Spectral Norms** or **RMS Norms** instead.
  - *Spectral Norm:* Max singular value.
  - *RMS Norm:* A computationally cheaper proxy for Spectral Norm (often scaled by $\sqrt{d_{in}/d_{out}}$).

9. Optimizer: Muon

- **Designed to Fix:** The disconnect between standard adaptive optimizers (like Adam) and the true spectral geometry of weight matrices in large Transformers.
- **Mechanism:** It forces weight updates to orthogonalize or respect the spectral structure (often using Newton-Schulz iterations, though the notes focus on the norm aspect).
- **Key Observation:** For mini-batch size 1, gradients are Rank-1 (uv^T). The sign of a Rank-1 matrix is also Rank-1.
- **Practice:** Shown to outperform Adam and Shampoo in "speedruns" (e.g., NanoGPT training).

Summary Concept Map

Like how it creates these different phases to attack when studying.

1. **Geometry (SVD)** $\rightarrow$ Determines Max LR (Phase 1)
2. **Geometry** $\rightarrow$ Momentum fixes oscillations (Phase 2)
3. **Scaling Laws (Width)** $\rightarrow$ Dictates Initialization (HW3 Prob 1) (Phase 3)
4. **Scaling Laws + Geometry** $\rightarrow$ μP (Stable Hyperparams) (Phase 3)
5. μP **+ Spectral Norms** $\rightarrow$ **Muon** (Next-Gen Optimizer) (Phase 4)

Turn this roadmap into an interactive study guide.

For each major concept:
1. Ask me one short diagnostic question to check whether I truly understand it.
2. After I answer, tell me whether my response reveals a missing prerequisite or misconception.
3. Only then suggest what I should study next.

Do not explain the material unless I explicitly ask for an explanation.

Wanted to see if gemini understands the topic at hand in depth. It does, as it creates a nice flow to understand the concepts as seen below.

Show thinking

This interactive format is excellent for solidifying these dependencies. I will guide you through the map one concept at a time.

Phase 1: The Geometric Foundation

Prerequisite Check: Linear Algebra & Gradients

Concept 1: Loss Landscape Geometry We must establish the "speed limit" of optimization before we discuss how to reach the destination.

Diagnostic Question: In a simple quadratic loss landscape (like a valley), if the curvature becomes "sharper" (steeper sides), does the maximum stable learning rate for Gradient Descent **increase** or **decrease**?

Bonus: Which specific property of the data matrix (or Hessian) mathematically quantifies this "sharpness"?

(Reply with your answer, and I will assess your readiness for the next concept.)

Diagnostic question(s) directly tests prerequisite knowledge (stability of GD) rather than surface familiarity with optimizers.

The max stable learning rate for GD decreases as the curvature becomes sharper, since steeper directions make gradient descent overshoot more easily.

For the bonus, sharpness is quantified by the largest eigenvalue in the Hessian/spectral norm/Lipschitz constant of the gradient

Show thinking

Spot on. High curvature (large λ_{max}) acts as a bottleneck, forcing you to take small steps to avoid overshooting (instability).

Evaluation is correct, but fairly permissive - it doesn't push on formal stability bounds (i.e. exact dependence on lambda_max)

Concept 2: Implicit Regularization

Prerequisite Check: Singular Value Decomposition (SVD)

Now, let's look at what happens when the landscape is "flat" in some directions—which is common in over-parameterized Deep Learning models where there are many possible solutions that achieve zero loss.

Diagnostic Question: If we initialize weights at zero and run Gradient Descent on a linear model with many valid solutions, GD doesn't pick a solution randomly.

Does GD prefer to build the weights using directions corresponding to **large** singular values or **small** singular values? (Hint: Think about which directions have the steepest gradients.)

GD prefers directions corresponding to larger sv's, since those directions have steeper gradients. Therefore, the parameters grow faster along them compared to directions with small sv's

Exactly. This is the essence of **Implicit Regularization**: GD is "lazy." It latches onto the strongest signals (large singular values) first and only bothers with the subtle details (small singular values) if forced to run for a long time.

Phase 2: Dynamics and Efficiency

Prerequisite Check: Statistics (Expectation/Variance) & Non-convexity

We now know *how* GD moves (following curvature), but calculating the exact gradient over millions of data points is too slow. We switch to **Stochastic Gradient Descent (SGD)**.

Concept 3: Stochastic Gradient Descent (SGD) SGD approximates the true gradient using a mini-batch. This introduces "noise"—the estimate is rarely exactly right, though it is right *on average*.

Diagnostic Question: In classical convex optimization, noise is generally bad (it prevents you from settling exactly at the bottom). Good conceptual check for implicit bias of GD that matches linear-model results discussed in lecture.

But in Deep Learning (non-convex landscapes), this noise is actually a **feature**, not a bug. specifically regarding **saddle points** (areas where the gradient is zero but it's not a minimum).

Why does the "noise" in SGD make it better at escaping saddle points than full-batch Gradient Descent?

What assumption does this concept rely on that would break in a different training regime?

I tried to shift from theory to application to see if gemini's intuition works on theoretical/applied boundaries.

Show thinking

The critical assumption is that the **batch size is small enough** to maintain a high variance (noise) in the gradient estimate.

This breaks in the **Large-Batch (or Full-Batch) Regime**. As your batch size grows, the gradient estimate approaches the "true" gradient, and the noise converges to zero. Without that noise, the optimizer recovers the behavior of deterministic Gradient Descent and can get stuck in saddle points (or take exponentially long to escape them).

This answer discusses SGD noise and batch size, which is relevant to non-convex optimization, but misses the key behind implicit bias in quad/linear models (i.e. linearity, smoothness, grad flow, etc).

Concept 4: Momentum

Prerequisite Check: Physics (Velocity/Inertia)

Now we have a noisy optimizer (SGD) that can escape saddle points, but it has a new problem: it's terrible at navigating "narrow valleys" (areas with high curvature in one direction and low curvature in another).

Diagnostic Question: In a narrow valley, standard SGD tends to oscillate wildly across the steep sides while making very slow progress down the gentle slope toward the minimum.

How does adding a **history parameter** (momentum) physically stop these oscillations? (Think about what happens to the "velocity" vector when the gradient direction keeps flipping back and forth).

Based on my answers so far, what should I study next before attempting the optimizer homework, and why?

Show thinking

Since you skipped the momentum check, I will briefly close that loop so you aren't missing a piece: **Momentum** acts like a physical ball gaining speed. It averages out the "zig-zag" oscillations (canceling them out) while accumulating velocity in the consistent direction down the valley.

Next Step: Phase 3 - Scaling & Feature Learning

Why this is next: You should study this next because **Homework 3, Problem 1** is entirely based on this concept. The problem asks you to calculate "Expected squared RMS norm" and find constants to multiply W by. These are the mathematical mechanics of **Initialization** (specifically Xavier/He initialization).

Strong learning-order recommendation: ties variance propagation to upcoming hw rather than proposing generic "review".

If you don't understand how variance propagates through a matrix multiplication, you cannot solve Problem 1.

Diagnostic Question: Assume you have a linear layer $y = Wx$.

- Input x has dimension d and elements with variance 1.
- Weights W are initialized from a standard Gaussian (variance 1).

If you do **not** scale the weights (i.e., you leave them as standard Gaussian), what happens to the variance of the output y as the dimension d gets very large?

(Does it stay at 1, go to 0, or explode to infinity?)

Intentionally quit interacting with gp3. At this point, I wanted to avoid turning gp3 into a tool rather than a diagnostic learning scaffold.